Trajectory-Based Vulnerability Response

A GyroAuth Applied Model for Post-Login Security Monitoring

Gyro Logic Lab

2026

Author: Gyro Logic Lab
Repository: https://github.com/gitGyro-Dev/gyroauth
Applied PoC: examples/trajectory_vulnerability_response/

Abstract

As AI-assisted vulnerability discovery advances, security models that rely solely on preventing or patching known vulnerabilities may become insufficient. This paper proposes **Trajectory-Based Vulnerability Response** as an applied GyroAuth model for post-login security monitoring. The central claim is that an exploit may consist of individually valid events while forming an impermissible and unstable operational trajectory. In this model, a vulnerability is interpreted as a structural weakness that allows an impermissible trajectory to be accepted as normal execution. The proposed framework consists of three layers: **Boundary / Negative Definition** for prevention, **Trajectory Detection** for post-entry anomaly detection, and **Stability Response** for containment, re-authentication, isolation, and recovery. A minimal proof-of-concept reconstructs session trajectories from synthetic access logs, evaluates deviations in sequence, velocity, context, scope, volume, permission, and destination, computes a stability value from weighted deviation, and maps the resulting state to staged responses. This work does not attempt to discover vulnerabilities or reproduce exploits; rather, it demonstrates how GyroAuth can extend authentication from login-time identity verification to continuous trajectory convergence monitoring.

Keywords: GyroAuth; Gyro Logic; Trajectory Detection; Vulnerability Response; Post-Login Security; Stability Response; State Convergence; Cybersecurity; Authentication; Anomaly Detection

---

1. Introduction

The increasing capability of AI systems to assist vulnerability discovery raises a practical question for security architecture. If unknown weaknesses can be found faster than they can be patched, security cannot rely only on the assumption that every vulnerability will be identified and closed before exploitation.

This does not imply that prevention, patching, input validation, and access control are unnecessary. They remain essential. However, modern systems also require a complementary layer: a method for detecting and responding to abnormal operational trajectories after an event has passed through normal processing paths.

Many attacks do not appear as obviously invalid events. A login may succeed. An API request may return 200 OK. A database query may complete. A file download may be allowed. Each event may be individually valid. The abnormality appears not in a single event but in the sequence, speed, scope, context, and destination of the events.

This paper therefore proposes the following core distinction:

Event validity is not trajectory stability.

Or, more compactly:

valid events != stable trajectory

This paper develops this distinction as a GyroAuth applied model called **Trajectory-Based Vulnerability Response**.

---

2. Background: GyroAuth

GyroAuth is an application-layer framework built on Gyro Logic and implemented in relation to GyroOS. Its core definition is:

Authentication = Stability-based Selection over State Convergence

In ordinary authentication systems, authentication is often treated as a point-in-time decision. A user provides credentials, a token is issued, and the session is treated as authenticated.

GyroAuth instead treats authentication as a state process. It observes whether a multi-dimensional state converges toward an expected identity trajectory, and then selects an authentication decision based on stability, deviation, operator response, and history.

This paper extends that idea from login-time identity verification to post-login session monitoring. The question is no longer only:

Is this user authenticated?

but also:

Does this post-login session trajectory remain stable within the expected identity, permission, and opera

This extension does not redefine Gyro Logic. It does not replace GyroOS. It is an applied GyroAuth note for post-login security monitoring.

---

3. Vulnerability as Impermissible Trajectory

A vulnerability is often described as a weakness in software, configuration, system design, or operation that can be exploited by an attacker. In the present model, the definition is sharpened in trajectory terms:

A vulnerability is a structural weakness that allows an impermissible trajectory to be accepted as normal

This definition emphasizes that a vulnerability is not merely a bug. A bug is a failure to behave as expected. A vulnerability is a failure to preserve security boundaries, state-transition constraints, or operational assumptions.

Examples include:

- data being interpreted as executable command,

- an unauthenticated state entering an authenticated trajectory,
- a general user trajectory crossing into an administrative trajectory,
- a resource boundary allowing access to another user's data,
- a session continuing normally after abnormal data movement.

In GyroAuth terms, vulnerability response must therefore consider not only whether an event is accepted, but also whether the resulting trajectory remains stable.

---

4. Attack as Valid Events Forming an Unstable Trajectory

An attack does not always appear as a single invalid event. In many practical cases, the attacker uses valid operations in an abnormal sequence.

For example:

login
-> api_access
-> bulk_download
-> external_transfer

Each event may be successful. The login may be valid. The API may be reachable. The download function may exist. The external transfer may be performed by an allowed endpoint. Yet the trajectory as a whole may be impermissible.

Let a session trajectory be defined as:

$$T_s = (e_1, e_2, \dots, e_n)$$

where each e_i is an event in session s.

Event validity may hold:

$$\forall e_i \in T_s,\ \text{allow}(e_i) = \text{true}$$

However, trajectory stability may still fail:

$$\text{Stable}(T_s \mid C_s) < \theta$$

where C_s denotes the context of the session, including user, role, device, time, location, resource, and operational history.

This is the central security claim of this paper:

All events may be allowed, but the trajectory may still be impermissible.

---

5. Three-Layer Model

The proposed model consists of three layers, as shown in Figure 1.

Trajectory-Based Vulnerability Response is structured into three layers:

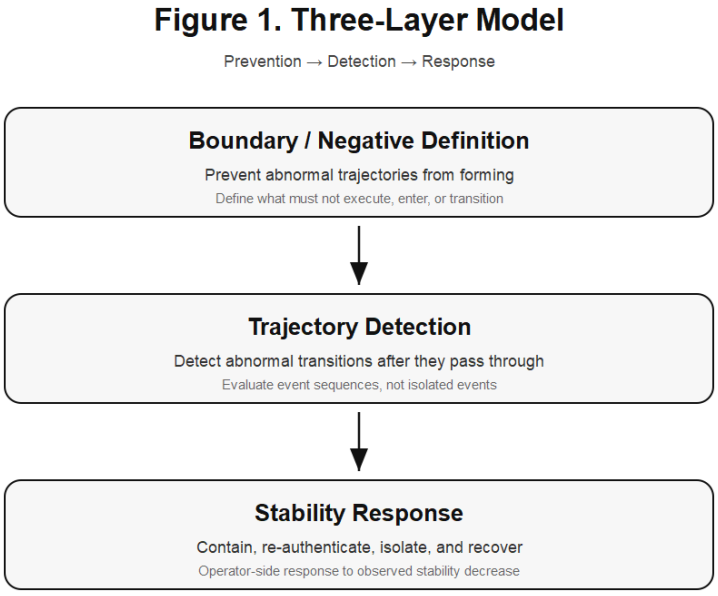

Figure 1: Three-Layer Model: prevention, detection, and response.

Boundary / Negative Definition
Trajectory Detection
Stability Response

These correspond to prevention, detection, and response.

5.1 Boundary / Negative Definition

Boundary / Negative Definition is the preventive layer.

Security design is not only the definition of what a system can do. It also requires defining:

- what the system must not execute,
- which state transitions must not occur,
- which inputs must not enter command space,
- which roles must not cross resource boundaries,
- which uncertain states must not reach critical operations.

In this sense, Boundary / Negative Definition is a way to prevent abnormal trajectories from forming.

Examples include:

Unauthenticated -> Admin API : disallowed
User role -> All resources : disallowed
Input data -> SQL command space : disallowed
Unknown device -> Bulk export : disallowed or challenged
Unstable session -> External send : disallowed or restricted

This is prevention. It reduces the space in which an abnormal trajectory can be formed.

5.2 Trajectory Detection

Trajectory Detection is the post-entry detection layer.

Since no boundary is perfect, some abnormal trajectories may pass through ordinary controls. Trajectory Detection reconstructs event sequences from logs and evaluates whether they remain stable within the expected context.

Typical deviation categories include:

- sequence deviation,
- velocity deviation,
- context deviation,
- scope deviation,
- volume deviation,
- permission deviation,
- destination deviation.

The detection target is not only the event. It is the event sequence.

Detection target = trajectory, not isolated event

5.3 Stability Response

Stability Response is the response layer.

Detection alone is not defense. When trajectory deviation is observed, the system must connect detection to staged control actions.

Possible responses include:

- continue,
- observe,
- challenge,
- restrict,
- isolate,
- recover.

In GyroAuth terms, Stability itself does not decide. Stability is an observed state variable. The operator-side response function maps observed stability and deviation to actions.

$$R = \rho(S, \Delta, C)$$

where S is observed stability, Delta is deviation, C is context, and rho is the operator-side response mapping.

---

6. Architecture for Existing Systems

The proposed model can be added to existing systems without immediately rewriting the application core. A minimal architecture is:

Existing Logs
-> Slice Normalizer

-> Trajectory Builder
-> Stability Evaluator
-> Deviation Detector
-> Response Controller

The overlay architecture is summarized in Figure 2.

The system first collects logs and events from the existing environment. These may include application logs, API gateway logs, authentication logs, authorization logs, database audit logs, file access logs, and session logs.

The Slice Normalizer converts these logs into a common event representation:


```
{
 "timestamp": "2026-05-21T10:15:30+09:00",
 "user_id": "user_123",
 "session_id": "sess_abc",
 "device_id": "dev_xyz",
 "source_ip": "203.0.113.10",
 "geo": "JP",
 "event_type": "api_access",
 "endpoint": "/api/export",
 "role": "user",
 "resource_type": "customer_data",
 "resource_id": "customers/*",
 "data_volume": 10485760,
 "status": "success"
}
```


The Trajectory Builder groups events by session, user, device, resource, or time window. The Stability Evaluator computes deviations from expected operational trajectories. The Response Controller maps the result to response actions through existing control points such as IAM, MFA, API Gateway, rate limiter, SIEM, or session manager.

---

7. Proof of Concept

A minimal proof-of-concept was implemented under:

examples/trajectory_vulnerability_response/

The PoC uses synthetic JSONL access logs and contains three sessions:

1. normal session,
2. suspicious session,
3. attack-like session.

The PoC follows the flow:

Logs -> Slice -> Trajectory -> Deviation -> Stability -> Response

---

7.1 Scenario

The normal session is:

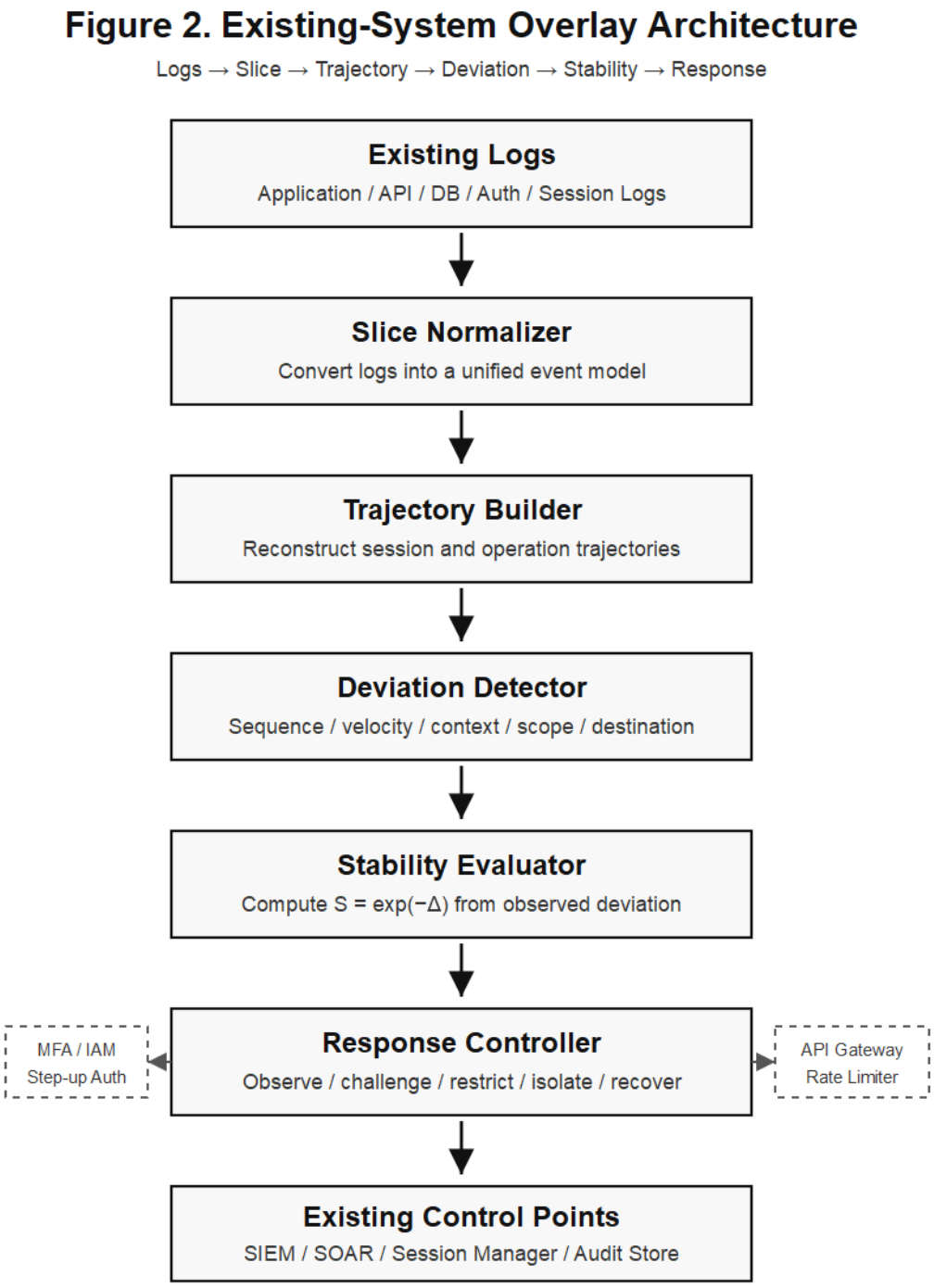

Figure 2: Existing-system overlay architecture for trajectory-based monitoring.

login -> view_dashboard -> search -> view_record -> logout

The suspicious session is:

login -> api_access -> export

The attack-like session is:

login -> api_access -> bulk_download -> external_transfer

All events are marked as success. This is intentional. The PoC demonstrates that successful events may still form an unstable trajectory.

7.2 Deviation Categories

The PoC evaluates the following deviations:

Deviation	Meaning
sequence	unusual event order
velocity	too many events in a short time window
context	unknown device, unusual network, unusual region, or unusual time
scope	access beyond normal resource scope
volume	unusual data volume
permission	role/resource mismatch or privilege-boundary anomaly
destination	external transfer or unknown destination

7.3 Stability Calculation

The PoC uses a deliberately simple model:

$$\Delta(T_s) = w_{\text{seq}}\Delta_{\text{seq}} + w_{\text{vel}}\Delta_{\text{vel}} + w_{\text{ctx}}\Delta_{\text{ctx}} + w_{\text{scope}}\Delta_{\text{scope}} + w_{\text{vol}}\Delta_{\text{vol}} + w_{\text{perm}}\Delta_{\text{perm}} + w_{\text{dest}}\Delta_{\text{dest}}$$

Stability is computed as:

$$S(T_s) = \exp(-\Delta(T_s))$$

This is an implementation model for the PoC. It is not a redefinition of Gyro Logic.

7.4 Response Mapping

The PoC maps stability to staged response states:

Stability	Auth State	Response
S >= 0.85	AUTH_STABLE	continue
S >= 0.65	RECONVERGING	observe
S >= 0.45	AUTH_CHALLENGE	challenge
S >= 0.25	AUTH_RESTRICTED	restrict
S < 0.25	AUTH_ISOLATED	isolate

7.5 Results

The PoC produced the following results:

Session	Delta	Stability	Auth State	Response
normal	0.000	1.0000	AUTH_STABLE	continue
suspicious	0.475	0.6219	AUTH_CHALLENGE	challenge
attack-like	2.705	0.0669	AUTH_ISOLATED	isolate

The stability results are summarized in Figure 3.

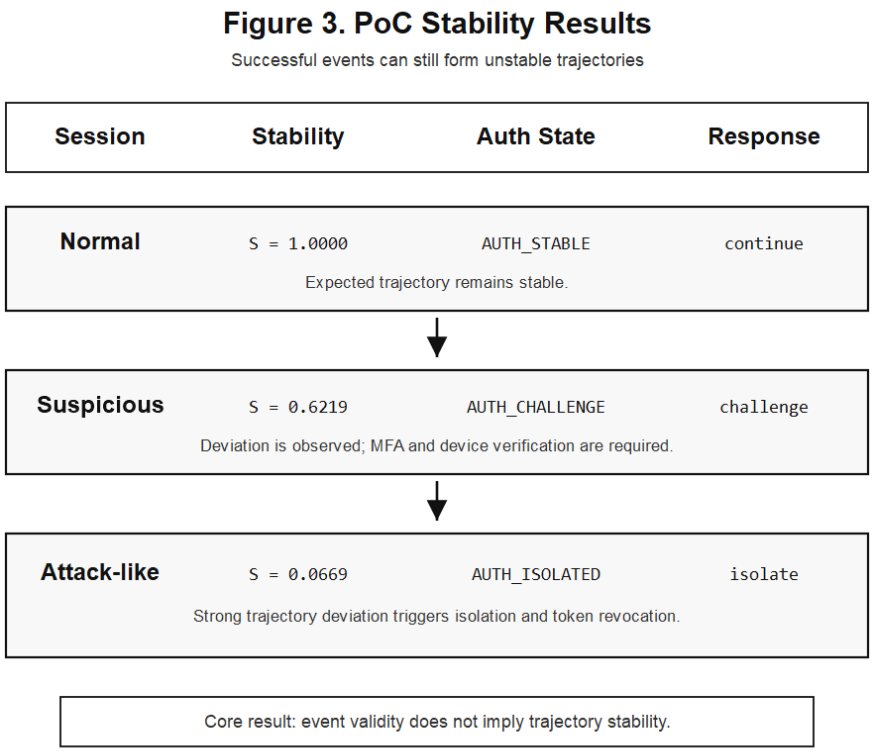

Figure 3: PoC stability results for normal, suspicious, and attack-like sessions.

The normal session remained stable. The suspicious session triggered a challenge response, requiring MFA and device verification. The attack-like session triggered isolation, including session isolation, token revocation, external transfer blocking, audit log preservation, and administrator notification.

This demonstrates the main claim of the PoC:

Successful events can still form an unstable trajectory.

---

8. Discussion

The proposed model is related to existing security monitoring concepts such as intrusion detection, SIEM, EDR, and user/entity behavior analytics. However, the emphasis here is different. The model does not only classify isolated events as normal or abnormal. It reconstructs event sequences as trajectories and evaluates whether they remain stable within identity, permission, resource, and operational context.

The model is also compatible with existing systems. It can begin as passive monitoring using logs, then progress toward alerting, step-up authentication, dynamic restriction, and session isolation.

This staged approach is important because deviation does not always mean attack. A user may travel, change devices, perform legitimate bulk work, or respond to an emergency. Therefore, Stability Response should not always be immediate termination. It should be a staged operator-side response.

---

9. Limitations

This paper presents a conceptual model and a minimal PoC. It has the following limitations:

- the PoC uses synthetic logs,
- no real-world dataset evaluation is performed,
- thresholds and weights are illustrative,
- no production intrusion detection claim is made,
- false positives require operational tuning,
- response actions require integration with real control points,
- the model does not replace patching, secure coding, access control, or existing security tools.

The purpose of the PoC is not to reproduce an exploit. It is to demonstrate that individually successful events can form an unstable trajectory and that such deviation can be mapped to staged response.

---

10. Conclusion

This paper proposed Trajectory-Based Vulnerability Response as an applied GyroAuth model for post-login security monitoring. The model begins from the observation that a vulnerability exploit may consist of individually valid events while forming an impermissible operational trajectory.

The proposed framework consists of three layers:

Boundary / Negative Definition = prevention
Trajectory Detection = post-entry abnormal transition detection
Stability Response = containment, re-authentication, isolation, and recovery

The minimal PoC demonstrates that successful events can be reconstructed as session trajectories, evaluated for deviation, converted into stability values, and mapped to staged response actions.

The central conclusion is:

Event validity is not trajectory stability.

For AI-assisted vulnerability discovery and increasingly complex systems, vulnerability response should evaluate not only whether events are allowed, but also whether the trajectory formed by those events remains stable.

Conflict of Interest

The author declares no competing interests.

References

1. Gyro Logic Lab, Gyro Logic: A Stability-Based Framework for Representation Under Intrinsic Deviation, Jxiv, DOI: https://doi.org/10.51094/jxiv.4159
2. Gyro Logic Lab, Gyro Logic: A Stability-Based Framework for Representation Under Intrinsic Deviation Japanese translation, Jxiv, DOI: https://doi.org/10.51094/jxiv.4597
3. Gyro Logic Lab, GyroAuth, GitHub repository: https://github.com/gitGyro-Dev/gyroauth
4. Gyro Logic Lab, Trajectory-Based Vulnerability Response PoC, GitHub repository path: https://github.com/gitGyro-Dev/gyroauth/tree/main/examples/trajectory_vulnerability_response